NYC Yellow Taxi Fare Prediction (Q1 2018)

Introduction

This report documents the analysis of the Q1 2018 New York City Yellow Taxi dataset, which involved over 305k records. The project focused on developing a robust methodology to manage significant data quality issues by cleaning and feature engineering steps to derive meaningful features, and build a high-performance predictive model for the fare_amount.

Business Problem

The primary business problem was addressing the lack of **reliable data quality and consistency** in the raw taxi trip records, which prevents accurate analysis of pricing and trip dynamics comprehensive.

The ultimate goal was to **accurately predict the trip fare amount** to enable better revenue management, pricing strategies, and urban transport analytics.

Data Analysis Workflow

The project followed a standard, systematic data science workflow comprehensive understanding, preparation (cleaning and engineering), modeling, and evaluation.

1. **Data Understanding**
   This step focused on loading the raw data and identifying its initial structure, types, and quality issues.
   We load dataset, examine summary statistics, and check column types and missing value counts.
2. **Preparation (Cleaning/Engineering)**
   This is the most critical phase, where raw data inconsistencies were resolved and analytical features were created, ensuring the dataset complies with regulations.

   Transform raw data into a clean, feature-rich dataset ready for modeling.

2.1. Data Cleaning

- 2.1.1. **Datetime:** Converted tpep_pickup_datetime and tpep_dropoff_datetime from object to datetime format to enable time-based calculations. We also swapped pickup and drop-off datetimes for 169 rows with negative durations to ensure all trip times are positive.
- 2.1.2. **Trip Duration:** Imputed 3,050 extreme durations (>99th percentile of 61 min) with grouped means by location and time_bucket (new feature generated) to cap unrealistic values.
- 2.1.3. **VendorID:** Replaced 503 invalid VendorIDs (4 or 5) with grouped modes from valid entries, ensuring all values are in [1,2,6,7].
- 2.1.4. **Passenger Count**: Filled ~9,513 NaNs with grouped medians and clamped values to 1–6, removing invalid zeros.
- 2.1.5. **RatecodeID:** Filled ~9,513 NaNs with 99 to explicitly flag unknown rate codes per data dictionary.

2.1.6. **Store and Forward Flag:** Filled ~9,513 NaNs with 'N' and converted to boolean (Y=True, N=False) for consistent handling.
2.1.7. **Congestion Surcharge:** Set all values to 0, as the fee was not applicable in Q1 2018.
2.1.8. **Airport Fee:** Assigned ±1.25 for JFK/LGA zone trips based on total_amount sign, correcting others to zero.
2.1.9. **Improvement Surcharge:** Standardized to ±0.3 based on total_amount sign to align with post-2015 policy requirements.
2.1.10. **MTA Tax:** Set to ±0.5 generally, but 0 for non-NYC (RatecodeID=4) or EWR trips to enforce exemptions.
2.1.11. **Trip Distance:** Flagged negatives, non-no-charge zeros, and outliers (>19.54 miles) as errors in distance recording. Replaced anomalies with imputed distances from grouped mean speeds (capped at 60 mph) times duration, setting min 0.01.
2.1.12. **Payment Type:** Corrected 1 invalid payment_type value outside the valid range and fixed 71 zero-total, 562 negative-total, 1,416 positive-total [3,6], 222 negative-total [3,6], 532 positive-total [4], and 3 zero-total [4] misclassified rows.
2.1.13. **Tip Amount**: Forced tip_amount to 0 for all cash payments (type=2) to match no tip policy.
2.1.14. **Amount Sign Consistency**: Negated or absolutized fare_amount, extra, mta_tax, tip_amount, tolls_amount, and improvement_surcharge to match total_amount sign.
2.1.15. **Amount Columns:** Identified positive (>99th percentile) and negative (<1st percentile) outliers separately in fare_amount, extra, tip_amount, and tolls_amount. Replaced anomalies with sign-specific means from grouped non-anomalous subsets to preserve financial logic.
2.1.16. **JFK Flat Rate:** Standardized fare_amount to ±52 for 7,389 RatecodeID=2 trips based on total_amount sign, leaving zeros unchanged.

2.2. Feature Engineering

2.2.1. **Trip Duration:** Derived trip_duration in minutes as the total seconds difference between dropoff and pickup datetimes.
2.2.2. **Time Bucket:** Categorized pickup_hour into discrete time_bucket labels ('morning', 'midday', 'evening_rush', 'evening', 'night') for temporal analysis grouping.
2.2.3. **Adjusted Total Amount:** Recomputed adj_total_amount by summing all cleaned fare components (fare + extra + mta_tax + tip + tolls + improvement + congestion + airport) for accurate totals.

3. Visualization:

In this step, we generated a series of exploratory plots using matplotlib, seaborn, and folium to make solid date to graphs and plots for ease of understand.
Key visuals included:

3.1. Horizontal barplot of the top 20 pickup-dropoff routes
3.2. Countplots and barplots for payment types
3.3. Grouped bars for time-bucket economics
3.4. Interactive Folium choropleths mapping spatial hotspots
3.5. Urban demand trends
3.6. Payment preferences

4. Modeling

In the modeling step, we developed a regression pipeline to predict fare_amount using 11 curated features from the cleaned dataset, starting with a stratified 50k-row sample for efficiency and splitting it 60/20/20 into train/validation/test sets to ensure balanced representation of fare extremes.

Preprocessing via scikit-learn's ColumnTransformer standardized numerical features and one-hot encoded categoricals, followed by hyperparameter tuning on a 20k subset using RandomizedSearchCV across three models Random Forest, XGBoost, and SVR yielding tuned configs.

Models were refit on full train sample data, validated (Random Forest R^2=0.91), and evaluated on test (RF best at R^2=0.93, MAE=0.86), demonstrating robust non-linear capture of drivers like distance and time, with implications for fare forecasting in urban transport.

Conclusion

In this project, we worked on a messy dataset of Q1 2018 NYC yellow taxi trips, featuring over 305,000 records. We began by cleaning and preparing the data for analysis, followed by feature engineering to get the dataset ready for exploratory data analysis (EDA) and modeling. In the visualization step, we evaluated key trends and customer preferences using suitable graphs and plots, uncovering patterns like heavy back-and-forth traffic in Midtown zones (e.g., 237↔236 accounting for over 2,000 trips), 71% of rides paid by credit card, and higher nocturnal fares averaging $13.93 compared to morning lows of $12.76. Finally, we built a regression pipeline to predict fare amounts, with results showing that Random Forest delivered the best performance—achieving an R^2 of 0.94 (explaining 94% of fare variance), a mean absolute error (MAE) of just $0.84 (under 6% of the average $13 fare), and RMSE of $2.64—outpacing XGBoost and SVR for accurate, real-world fare forecasting.